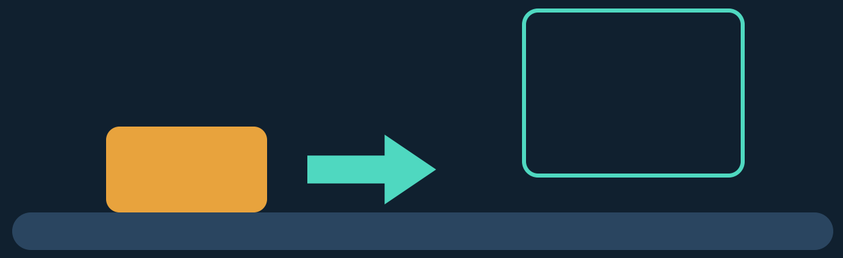
VTU BE FINAL YEAR PROJECT · INTERNET OF THINGS

MediDispense

Automated medicine delivery to the hospital bedside

[Student names] · [Guide name] · [Department, College]



WHY THIS PROJECT

Medicine delivery in a ward is still manual

Staff time

Nurses walk medicines from bed to bed, time that could go to patient care.

Room for error

A box left at the wrong bed or at the wrong time is easy to miss and hard to trace.

No live record

Doctors and patients cannot see where a prescribed medicine is right now.

Goal: a low-cost IoT system that takes a prescription from the doctor to the right bed automatically, with a record at every step.

HOW WE APPROACHED IT

Three decisions that kept it simple

Delivery

FIRST IDEA

A line-following robot

WHAT WE BUILT

A fixed belt with one pusher per bed. Nothing to navigate, so far fewer things can fail.

Verification

FIRST IDEA

QR scans of the medicine and the patient

WHAT WE BUILT

One button and one IR sensor per bed. The backend still supports scanning as an option.

Interface

FIRST IDEA

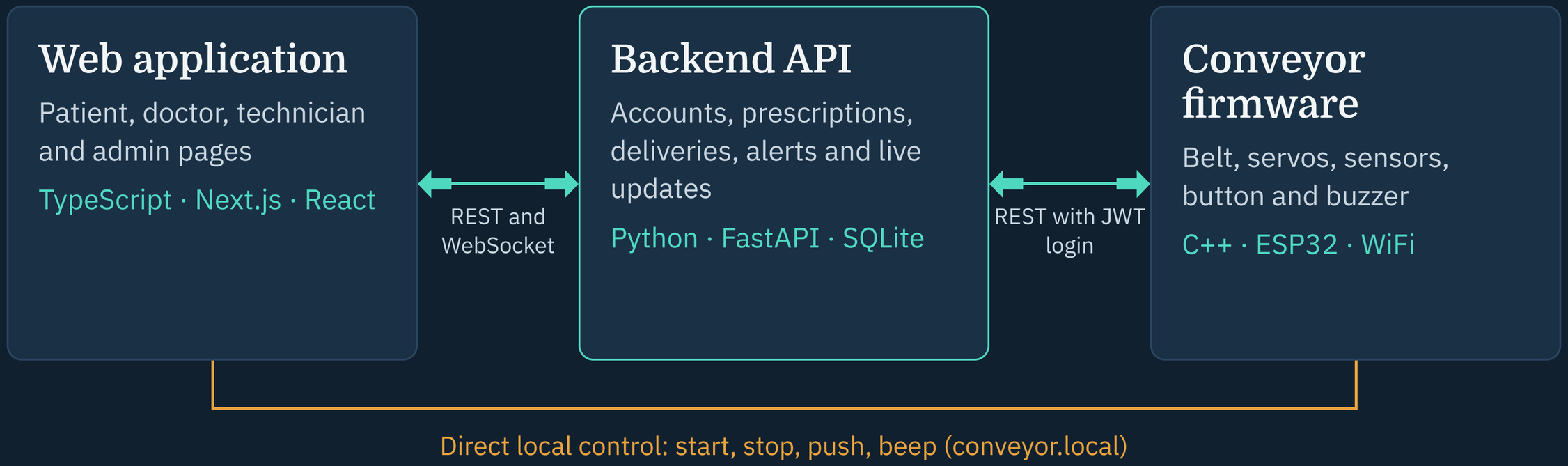
An Android app plus an admin website

WHAT WE BUILT

One web app for every role, usable from any browser with nothing to install.

SYSTEM ARCHITECTURE

Three components, one source of truth



HARDWARE

The components and what each one does

ESP32 DevKit

The brain, with built-in WiFi. Runs the station firmware.

L298N driver

Switches the belt motor and sets its speed.

Belt kit

Wood frame, rubber belt and drive motor. A 20 cm belt.

12 V adapter

Powers the motor driver. The ESP32 runs from USB.

2 × SG90 servo

A pusher arm at each bed that shoves the box into its chute.

2 × IR sensor

Sees the box arrive at each bed’s pusher line.

Push button

The operator confirms the box is loaded and starts the run.

Buzzer

Beeps when a job arrives and when the box is delivered.

HARDWARE IN DETAIL

Controller and belt drive

ESP32 DevKit

- Dual-core 32-bit microcontroller with built-in WiFi
- Runs the station firmware
- Reads the sensors and the button, drives the motor driver, servos and buzzer
- Logs in to the backend and reports every step

L298N motor driver

- ESP32 pins cannot power a motor, so this board switches the motor current
- ENA takes a PWM signal that sets the belt speed
- IN1 and IN2 set the direction
- Runs from the 12 V adapter, grounded with the ESP32

Belt kit and motor

- Wooden frame, rubber belt and pulleys
- A 20 cm belt, 6.5 cm wide
- A small DC gear motor turns the drive roller
- Runs slowly and steadily so the box does not slide

HARDWARE IN DETAIL

Sensing the box and pushing it

IR obstacle sensor, one per bed

- An infrared LED sends out light and a receiver beside it watches for the reflection
- A box in front reflects the light back, and the output pin goes LOW
- Sensor A watches the Bed 1 line and sensor B the Bed 2 line
- The firmware needs 5 matching readings 10 ms apart, so a flicker is never a box
- Runs from 3.3 V, so its signal is safe for the ESP32 pin

SG90 servo, one per bed

- A small motor with built-in gears and position control
- The ESP32 sends a PWM pulse and the servo turns to that angle
- The arm rests at 0 degrees, swings to 90 degrees to push the box off the belt, then returns
- A plastic arm with a paddle is fixed to the servo horn
- Servo 1 pushes into Bed 1 and servo 2 into Bed 2, from opposite sides of the belt

HARDWARE IN DETAIL

Operator interface and power

Push button

- Wired between pin D15 and ground
- An internal pull-up keeps the pin HIGH until the button is pressed
- Debounced in software, so one press counts once
- Starts a delivery run. Pressed during a run, it is an emergency stop

Buzzer

- An active buzzer that sounds while pin D23 is HIGH
- Short beep: a job arrived, or the button was pressed
- Long beep: the box was delivered
- Three quick beeps: an error or a timeout

Power

- A 12 V adapter feeds the L298N and the belt motor
- The ESP32 runs from USB
- Its 3.3 V pin powers the sensors; its 5 V pin powers the servos and buzzer
- Grounds are shared so every signal has the same reference

HARDWARE

How it is wired to the ESP32

Device	ESP32 pin
Sensor A (Bed 1)	D4
Sensor B (Bed 2)	D32
Push button	D15
Buzzer signal	D23
Servo 1 (Bed 1)	D18
Servo 2 (Bed 2)	D19
L298N ENA, IN1, IN2	D25, D26, D27

Shared power rails

3V3: both sensors

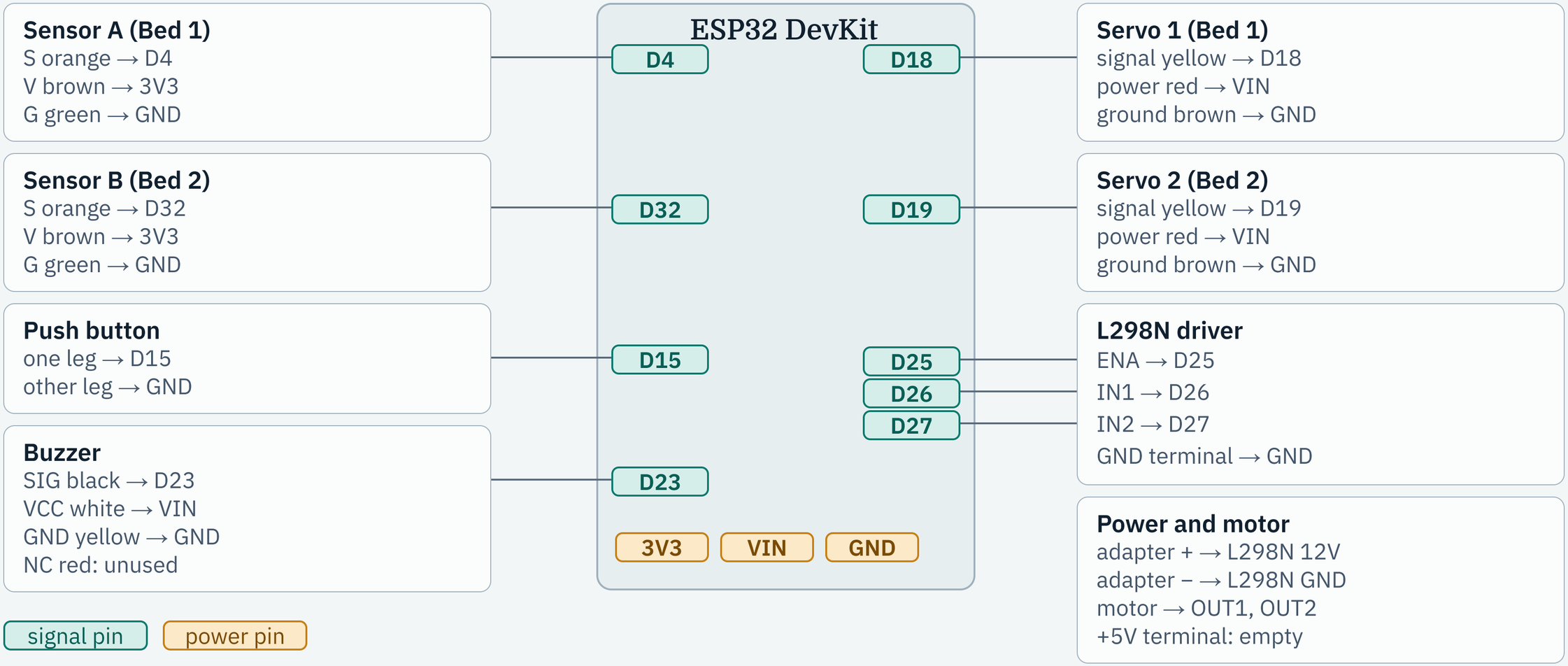
VIN, 5 V: both servos and the buzzer

Ground one: sensors, button, buzzer

Ground two: servos and the L298N

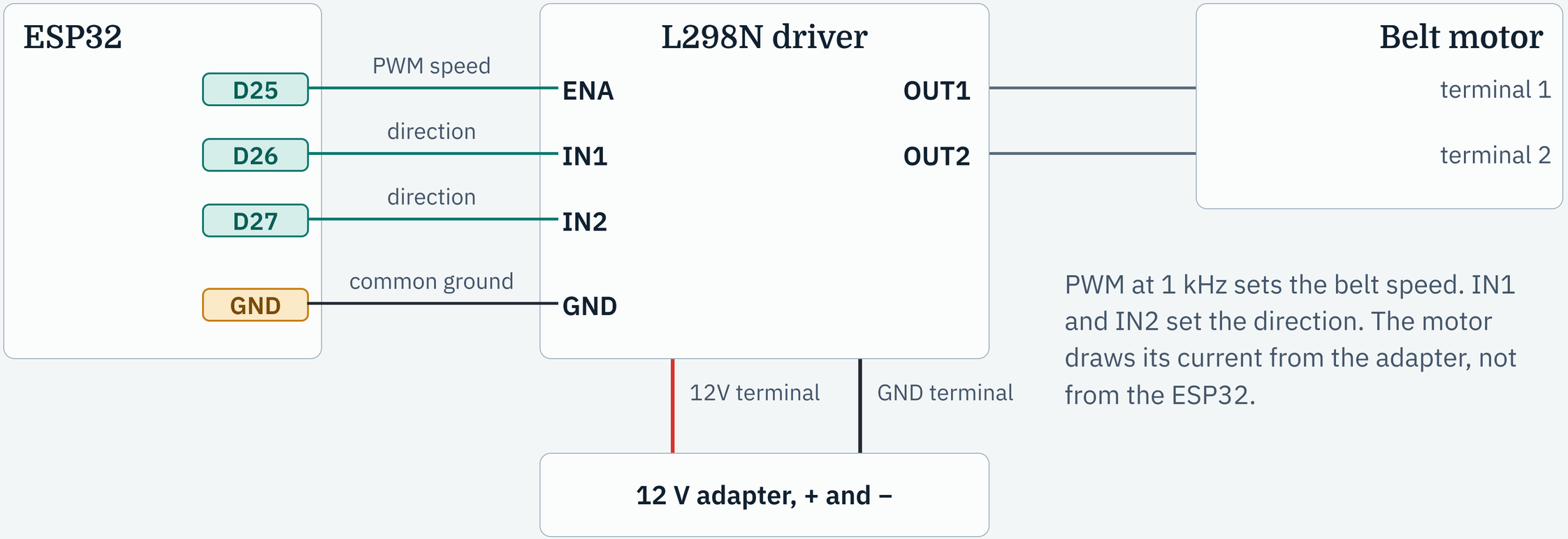
The motor runs on a separate 12 V adapter. Only the ground is shared.

Complete pin diagram



WIRING BY COMPONENT

Belt drive: ESP32, L298N and motor



PWM at 1 kHz sets the belt speed. IN1 and IN2 set the direction. The motor draws its current from the adapter, not from the ESP32.

WIRING BY COMPONENT

IR obstacle sensors

Sensor A, Bed 1

S, signal (orange)	D4
V, power (brown)	3V3
G, ground (green)	GND

Sensor B, Bed 2

S, signal (orange)	D32
V, power (brown)	3V3
G, ground (green)	GND

Shared: both sensors join the same 3V3 rail and the same ground (ground one). Only the signal wires have their own pins.

Reading: the pin is HIGH when clear and goes LOW when a box reflects the sensor’s infrared light.

WIRING BY COMPONENT

SG90 servo pushers

Servo 1, Bed 1

Signal (yellow)	D18
Power (red)	VIN
Ground (brown)	GND

Servo 2, Bed 2

Signal (yellow)	D19
Power (red)	VIN
Ground (brown)	GND

Shared: both servos join the VIN (5 V) rail with the buzzer, and ground two with the L298N. Each servo has its own signal pin.

Motion: the arm rests at 0 degrees, swings to 90 degrees to push the box off the belt, then returns.

WIRING BY COMPONENT

Push button and buzzer

Push button

One leg	D15
Other leg	GND

No power wire: the pin’s internal pull-up holds it HIGH until the button connects it to ground.

Buzzer

SIG, signal (black)	D23
VCC, power (white)	VIN
GND, ground (yellow)	GND
NC (red)	not used

Shared: the button and the buzzer’s ground join ground one with the sensors. The buzzer’s power joins the VIN rail with the servos.

FIRMWARE

How the conveyor decides what to do

Idle

Asks the backend for the next job every 4 seconds.

Loading

A job is waiting. The operator loads the box and presses the button.

Running

Belt runs until the right bed’s sensor sees the box. Then stop, push, beep, report.

Built-in safety

- Press the button during a run: emergency stop
- No box within 40 seconds: timeout, jam alert, delivery closed as failed
- Each bed has its own sensor, so a Bed 2 box ignores sensor A

FIRMWARE IN DETAIL

What the firmware runs on the ESP32

Network

Joins the WiFi, announces itself as conveyor.local through mDNS, and finds the backend by its computer name.

Backend client

HTTPClient and ArduinoJson: logs in, fetches the next job, dispatches, marks arrived or failed, and reports belt status. Every call has a timeout.

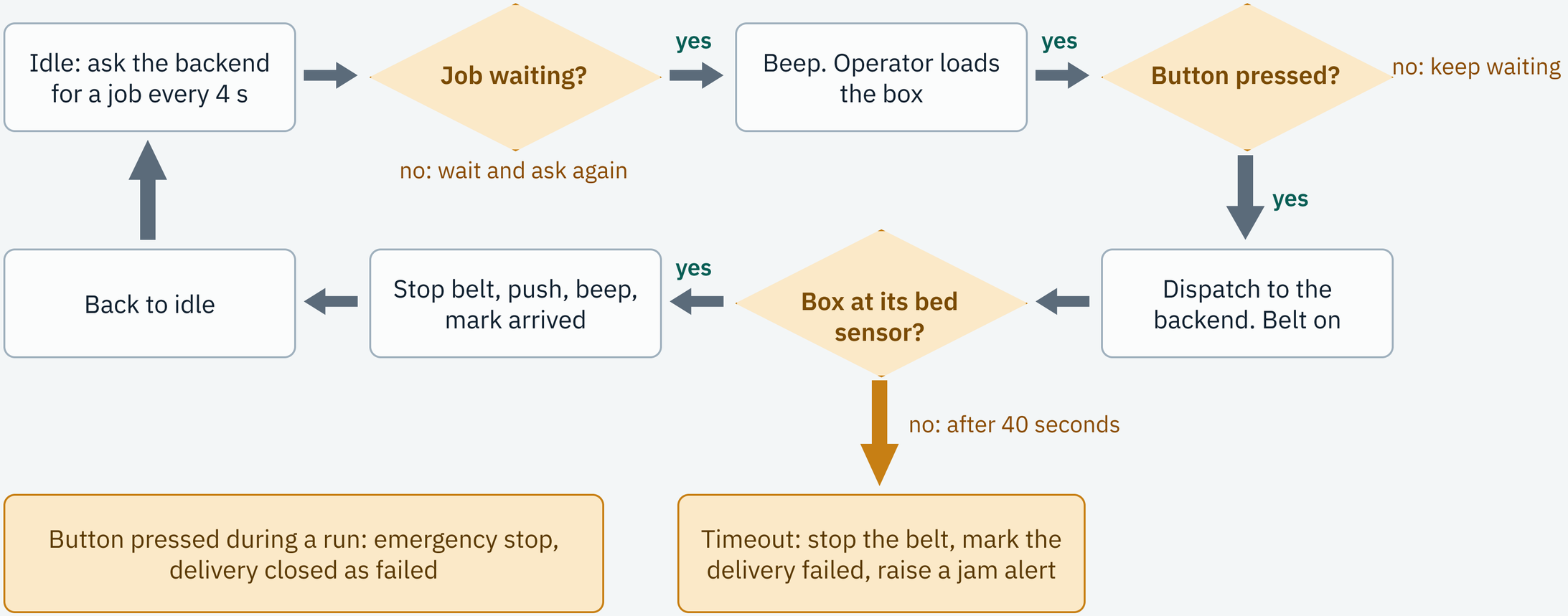
Local control server

A small web server: GET /status, and POST /start, /stop, /push and /beep. It allows cross-origin calls so the web page can use it.

Motor and inputs

PWM belt speed at 1 kHz, the ESP32Servo library for the pushers, and sensors and button read every 10 ms with debouncing.

The conveyor’s flowchart



SOFTWARE

The software stack

Backend

- Python 3.13
- FastAPI
- SQLAlchemy and SQLite
- JWT login
- WebSocket live updates
- pytest, 7 tests

Web application

- TypeScript
- Next.js 16
- React 19
- Tailwind CSS 4
- Axios
- Four role-based interfaces

Firmware

- C++ on the Arduino framework
- ESP32 core 3.3.11
- WiFi and HTTPClient
- WebServer and mDNS
- ESP32Servo, ArduinoJson

REST with JSON between components, WebSocket for live updates, and a JWT for every login.

SOFTWARE IN DETAIL

The backend: FastAPI and friends

What each library does

- **FastAPI:** defines the REST endpoints, with automatic request checking and interactive API docs
- **Uvicorn:** the server that runs it, with WebSocket support
- **Pydantic:** validates every request and response
- **SQLAlchemy:** database tables written as Python classes; SQLite now, MySQL-ready, with Alembic migrations
- **JWT and bcrypt:** signed login tokens and hashed passwords

What it exposes

- API groups: auth, users, patients, consultations, prescriptions, deliveries, conveyor, alerts and reports
- Every endpoint declares which roles may call it, and anyone else is refused
- WebSocket channels for the conveyor, alerts and prescriptions push live updates to the browser
- Seven automated tests run the whole flow with pytest

SOFTWARE IN DETAIL

The delivery API and its life cycle

Method	Endpoint	Called by
POST	/prescriptions	Doctor
GET	/conveyor/next-job	Conveyor
POST	/deliveries/{id}/dispatch	Conveyor, on the button
POST	/deliveries/{id}/mark-arrived	Conveyor, after the push
POST	/deliveries/{id}/receive	Patient
POST	/deliveries/{id}/fail	Technician or conveyor



SOFTWARE IN DETAIL

The web application

How it is built

- **Next.js and React:** each screen is a page made of components, written in TypeScript
- **Tailwind CSS:** styling, with light and dark modes
- **Axios:** sends the login token with every API call
- **Role-gated pages:** a user is redirected to the pages for their own role
- **WebSocket hook:** keeps dashboards live without refreshing

Key screens

- **Patient:** sign-up, find a doctor, consultations, delivery progress and “I received it”
- **Doctor:** availability, requests, prescribing and live delivery status
- **Technician:** belt status, alerts, deliveries in progress and live hardware control
- **Admin:** dashboard, staff, patients, prescriptions and reports

SOFTWARE

One web app, four kinds of user

Patient

Signs up with a bed number, requests a doctor, tracks the medicine and confirms receipt.

Doctor

Sets availability, accepts requests, writes prescriptions and watches each delivery live.

Technician

Monitors the belt and alerts, controls the conveyor directly and closes stuck deliveries.

Administrator

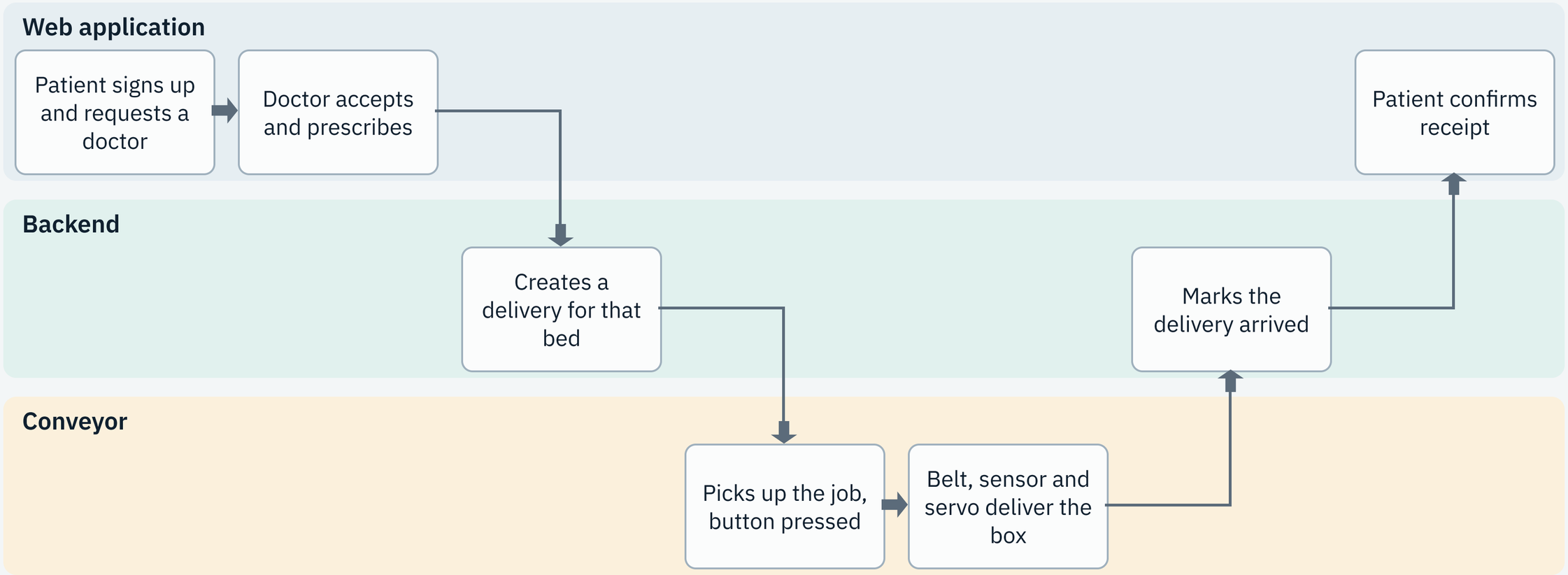
Manages staff accounts and oversees every record, delivery, alert and report.

WORKFLOW

One prescription, from doctor to bed

1. Patient signs up and requests a doctor
2. Doctor accepts and writes the prescription
3. Backend creates a delivery for that bed
4. Conveyor picks up the job and beeps
5. Operator loads the box and presses the button
6. Belt runs and the sensor at the bed stops it
7. Servo pushes the box into the bed’s chute
8. Delivery marked arrived. Patient confirms receipt

The complete project flow



Every step is recorded by the backend, and the dashboards update live.

RESULTS

What we tested

7

automated backend tests passing

5

clean box pushes confirmed on Bed 1

On the real machine

- Belt runs steadily and quietly
- Sensors react to a hand and to the box
- Sensor stops the belt, servo pushes the box off
- Button starts a run, buzzer beeps on delivery

The web app’s Start, Stop, Push and Beep buttons also command the real conveyor over WiFi.

RESULTS

Where the project stands

Working now

- Backend with accounts, prescriptions, deliveries and alerts
- Web app for patient, doctor, technician and admin
- Belt, sensors, button, buzzer and servo 1 verified on Bed 1
- Direct conveyor control from the web page

Finishing

- Run the final firmware for a full prescription-to-bed delivery
- Bed 2 trial on the belt (servo 2 and sensor B)
- Calibrate the servo angles on the mounted paddles
- Replace the default admin password

WHAT WE LEARNED

Problems we hit and how we solved them

The belt only buzzed at low motor speed	Removed the enable jumper and tuned the PWM to 1 kHz
The IR sensors’ signal wire was unreliable	Soldered wires straight onto the sensor pins
Seven parts, only two ground pins	Four shared power rails, grounds split by group
The board’s IP address kept changing	Found by name (conveyor.local), with an editable address

WHAT COMES NEXT

Future work

Near term

- A WiFi setup page, so the board can join a new network without reflashing
- A fixed network address for the server and the board
- Servo angles saved from the web page

Longer term

- More beds, with one pusher and sensor each
- Optional QR verification of medicine and patient, already supported by the backend
- Push notifications to phones
- HTTPS, device authentication and a MySQL deployment

MEDIDISPENSE

Thank you

Questions and suggestions are welcome

Code: github.com/SHIVAAKARTHIK (conveyor-backend, conveyor-admin-web)